

Reviewer Pack — Real Stable Rank Lower-Bounds AC^0 PARITY Circuit Size

Entry 3dac7381047e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 07:54:01 UTC

Real Stable Rank Lower-Bounds AC^0 PARITY Circuit Size

Entry ID: 3dac7381047e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 07:54:01 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): AC^0 Circuit Size for PARITY

Statement:

For any AC^0 circuit C computing PARITY on n bits, the real stable rank of the matrix M_C representing C 's linear transformation satisfies $\text{rank_stable}(M_C) \geq \Omega(\log n)$.

Rationale (proposer's reasoning):

Real stable rank captures the minimal number of rank-1 matrices needed to approximate a linear transformation. PARITY's linearity imposes structural constraints on M_C , which may translate to lower bounds on its stable rank, reflecting inherent complexity in AC^0 representations.

Taxonomy category: AC^0 _PARITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4845e18ab2e55319

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=2.1s

5.1 Generated Python source

```
import random
import math

def generate_truth_table(n):
    return [(i >> j) & 1 for j in range(n)] for i in range(2**n)]

def walsh_hadamard_transform(table):
    n = len(table)
    transform = table[:]
    s = 1
    while s < n:
        for k in range(s):
            for i in range(k, n, 2 * s):
                j = i + s
                a, b = transform[i], transform[j]
                transform[i] = a + b
                transform[j] = (a - b) * math.sqrt(2)
            s *= 2
    return [x / math.sqrt(n) for x in row] for row in transform]

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(matrix):
    n = len(matrix)
    augmented_matrix = [row[:] + [1 if i == j else 0 for j in range(n)] for i, row in enumerate(matrix)]
    for i in range(n):
        pivot = augmented_matrix[i][i]
        if pivot == 0:
            return None
        for j in range(i, n * 2):
            augmented_matrix[i][j] /= pivot
        for j in range(n):
            if j != i:
                factor = augmented_matrix[j][i]
                for k in range(n * 2):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
    return [row[n:] for row in augmented_matrix]
```

```

def rank_stable(matrix):
    reduced_matrix = gaussian_elimination(matrix)
    if reduced_matrix is None:
        return 0
    rank = sum(1 for row in reduced_matrix if any(row))
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10
    instances_tested = 30
    conjecture_holds = True
    counterexample = ""

    for _ in range(instances_tested):
        truth_table = generate_truth_table(n)
        M_C = walsh_hadamard_transform(truth_table)
        rank_stable_M_C = rank_stable(M_C)
        if rank_stable_M_C < 0.6 * math.log2(n):
            conjecture_holds = False
            counterexample = f"rank_stable(M_C) = {rank_stable_M_C} < 0.6 * log2({n})"

    return {
        "metric_name": "rank_stable",
        "metric_value": rank_stable_M_C,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5dcbe19c.py", line 96, in <module>
    result = run_trial(seed)
             ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5dcbe19c.py", line 76, in run_trial
    M_C = walsh_hadamard_transform(truth_table)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5dcbe19c.py", line 31, in walsh_hadamard
    transform[j] = (a - b) * math.sqrt(2)
                   ~^~
TypeError: unsupported operand type(s) for -: 'list' and 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to TypeError in Walsh-Hadamard transform calculation, preventing data collection | next: Fix the type error in the Walsh-Hadamard transform implementation by ensuring numerical operations on scalar values

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	106589
2	propose	ollama_remote	qwen3:8b	0	0	51269
3	preregistration	ollama_remote	qwen3:8b	0	0	9480
4	novelty	ollama_remote	qwen3:8b	0	0	19300
5	novelty	ollama_remote	qwen3:8b	0	0	7909
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19120
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23504
8	judge	ollama_remote	qwen3:8b	0	0	107265

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 344435 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/3dac7381047e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3dac7381047e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3dac7381047e.tar.gz` (if generated)